2025 webconf 分享

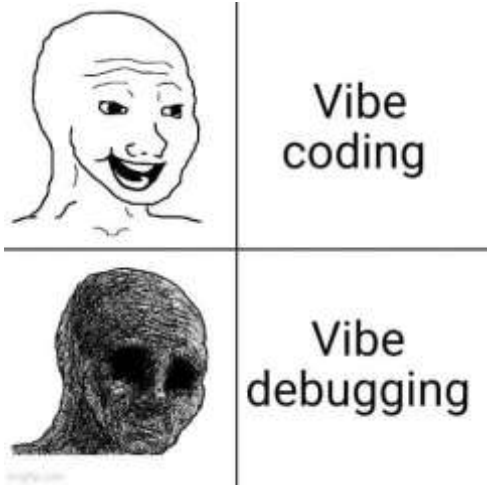
AI 只懂 React？Vue.js 也能 Vibe Coding！

研發部 Jenny 2026.02.12

第一章：
AI 時代下 Vue 開發者的挑戰

- 目前的 AI 生成工具（如 v0.dev, Bolt.new）多以 React 為預設
- 偏向輸出 useEffect 等 React 特有的 Hooks
- Vibe Coding 的表面與真相
  - 可以快速產出，但底層一堆錯誤
  - 累積一堆技術債

```
❌ TypeError: Cannot read property 'value'
❌ [Vue warn]: Invalid prop type
⚠️ Missing required prop: "user"
❌ Uncaught Promise rejection
```



第二章：
從 Vibe Coding 到 SDD

SDD 核心 — 雙重 SSOT 與閉環控制

- 什麼是 SDD？
  - 核心公式：Vibe Coding 的速度 + 結構化的意圖 = SDD
  - 先定義「意圖 (Spec)」，再由 AI 生成程式碼
- 核心原則：雙重 SSOT (單一真理來源)
  - Spec：意圖的真理來源（Source of Intent），定義「是什麼 (What)」。
    - 例如：這是一個使用者卡片，當角色是 Admin 時要顯示金邊。
  - Code：行為的真理來源（Source of Behavior），定義「怎麼做 (How)」。
    - 例如：這裡要用 if else 做判斷。

SDD 開發閉環

- 步驟拆解：
  - Spec (人)：定義 What & Why，流程的靈魂所在。
  - Generate (AI)：同時生成 Code + Test。
  - Verify (機器)：自動化驗收測試結果。
  - Sync (AI)：反向同步更新 Spec，解決文件與代碼不同步的問題。

第三章：
Vue SFC 之於 SDD 的獨家優勢

為何 Vue SFC 天然適合 SDD？

- 明確邊界：改樣式動 <style>，改邏輯動 <script>，AI 不易改錯地方。
- 結構化上下文：SFC 架構本質上是幫 AI 做好了「預處理」。

app.vue

```
<template>
<div id="app">
    {{ message }}
</div>
</template>

<script>
var app = new Vue({
    el: '#app',
    data: {
        message: 'Hello Vue!'
    }
})
</script>

<style lang="stylus">
body
    background white
    color gray
</style>
```

<spec> Custom Block

- 就像給 AI 的合約：比起註解，<spec> 標籤更具結構與定位。
- Zero Runtime Cost：利用 Vite Plugin 打包時移除標籤，不影響效能。

```
<script setup>
import { ref } from 'vue'

const message = ref('Hello World')
</script>

<template>
  <div>{{ message }}</div>
</template>

<style scoped>
div {
  color: blue;
}
</style>

<spec lang="md">
- 這是一個簡單的 Vue 元件，顯示一個藍色的 "Hello World" 訊息。
- 使用了 Vue 3 的 Composition API 來定義響應式變數 `message`。
- 樣式使用了 scoped，確保樣式只作用於此元件。
- `message` 的文字顏色設定為藍色。
</spec>
```

第四章：
實務策略與避坑指南

SDD 實務建議：分層策略

- 核心層：必須執行完整的 Spec -> Code -> Test -> Verify 閉環。
  - 例如：金流相關、權限控制、核心業務元件
- 業務層：可重用元件，簡化閉環為 Spec -> Code -> Sync。
  - 例如 ：可重用的 UI 元件、複雜的互動邏輯
- 展示層：直接 Vibe Coding 快速迭代。
  - 一次性的活動頁面、簡單的 Layout 、包裹容器 (wrapper or container)

Vue x AI 避坑指南

- Reactivity Loss：提醒 AI 在 Vue 3.4 以前解構 Props 會失去響應性。
- 版本混淆：明確禁止 Options API 關鍵字（如 this, data）。
- 資安風險：強制規定使用 dompurify 過濾 v-html。

先產出一份 AI 工作手冊

- 各 AI Agent 設定檔位置
  - Claude Code：讀取 CLAUDE.md。
  - Cursor：讀取 .cursorrules。
  - GitHub Copilot：讀取 .github/copilot-instructions.md。

- 規則撰寫建議
  - 使用強烈語氣：使用 ALWAYS 或 NEVER 比「建議」更有效。
  - 設定邊界：明確告訴 AI 什麼絕對不能碰（例如：NEVER use Options API）。
  - 保持精簡：規則太多 AI 反而會忽略，應專注於高風險的地雷區。

第五章：總結

工程師的核心價值

- 基本 - 一定要比 AI 更懂 Vue
- 審查 - 絕對不要 Accept all
- 驗證 - 要跑測試、實際操作
- 不要當 Prompt User，而要成為 AI Director

謝謝大家~